

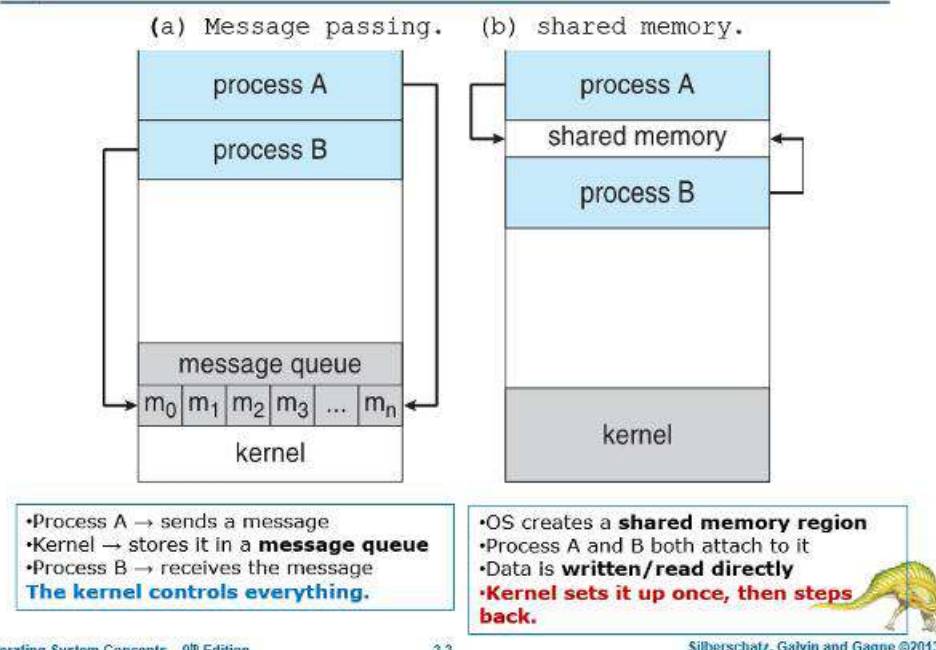
Mids Prep Material

Communications – Processes may exchange information, **on the same computer or between computers over a network**

- **Inter-process Communication (IPC):** Communication between processes on the same system
 - **Shared memory:** Processes share and access a common memory area. Like Two processes read/write the same data, example: Audio player reads it and plays sound
 - **Message passing:** Processes communicate by sending and receiving messages, example: A browser process sends a request to a print service.
- **Network Communication:** Communication between **processes on different systems**
 - **Sockets:** Used to send data over a network, example: A web browser talking to a web server
 - **Protocol stack:** Handles protocol stack management (TCP/IP), example: Ensures data arrives correctly over the internet.



Interprocess Communication



The kernel is the part of the OS that controls hardware resources and provides services to programs. It acts as a bridge b/w hardware and software (applications).

A **kernel module** is a **piece of operating system code** that can be **loaded into or removed from the kernel while the system is running** (adding features **only when needed**.)

- **kernel modules?** Independent components, Loaded/unloaded at runtime, Communicate via defined interfaces

- **Example:** USB driver loads only when device is plugged in
- Network module loads when network is used

Q1: Direct vs. Indirect Mode of IPC Message Passing

Direct and Indirect Communication in Message Passing

Direct Communication

In direct communication, each process that wants to communicate must explicitly name the recipient or sender of the communication.

- **Working:**
 - **Send:** `send(B, message)` — Process A sends a message specifically to Process B.
 - **Receive:** `receive(A, message)` — Process B receives a message specifically from Process A.
- **Properties of the Communication Link:**
 - **Automatic Establishment:** The link is created automatically by the OS as soon as one process sends a message to the other.
 - **Exclusivity:** A link is associated with exactly one pair of communicating processes.
 - **Single Path:** Between every pair of communicating processes, there exists exactly one link.
 - **Direction:** The link may be **unidirectional** (A to B only) or **bidirectional** (both can send and receive).

Indirect Communication

In indirect communication, messages are sent to and received from **mailboxes** (also referred to as **ports**). The system must allow processes to **create** a new mailbox, **send/receive** messages through it, and **destroy** the mailbox.

- **Working:**
 - Each mailbox has a **unique ID**.
 - Processes communicate only if they share a common mailbox.
 - **Primitives:** `send(A, message)` sends to mailbox A, and `receive(A, message)` receives from mailbox A.
- **Properties of the Communication Link:**
 - **Mailbox Dependent:** A link exists between two processes only if they share a common mailbox.
 - **Multiple Processes:** A single link (mailbox) may be associated with many different processes.
 - **Multiple Links:** Two processes may share several different communication links if they share multiple mailboxes.
 - Link may be unidirectional or bi-directional

Q2: How Computers Handle Data Transfer (I/O)

Computer systems handle I/O through coordination between **CPU, device controllers, and the operating system**. The CPU and I/O devices can work **concurrently** thus improving efficiency.

Each device is managed by a **device controller**, which has a **local buffer** to store data temporarily. The operating system maintains a **device-status table**, which stores information about each device (type, address, state). This helps the OS track and manage I/O operations.

Each device also has a **device driver**, which provides a uniform interface between the device controller and the kernel.

During I/O:

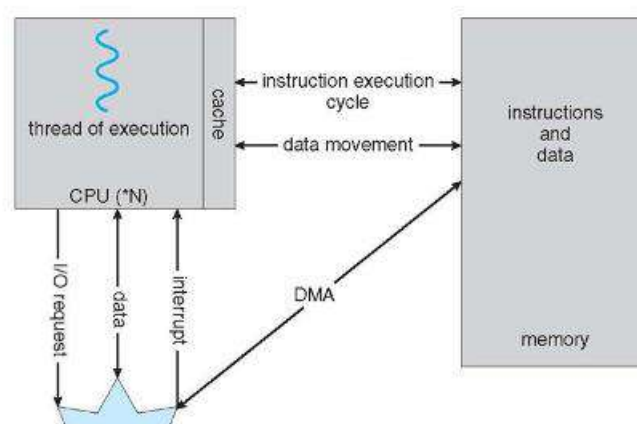
- Data is transferred between the device and the controller's buffer
- The CPU moves data between **main memory and the buffer**
- After completion, the controller sends an **interrupt** to the CPU

Computers manage this type of movement of data using three primary techniques. Each method offers a different balance of CPU involvement and speed.

- **Programmed I/O:**
 - The CPU is directly responsible for the data transfer.
 - It must constantly monitor (poll) the device to see if it is ready or not.
 - **Drawback:** This is time-consuming as it keeps the CPU busy in a "loop" instead of doing other work.
- **Interrupt-Driven I/O:**
 - Instead of the CPU waiting, the device signals the CPU using an **interrupt** when it is ready to transfer data.
 - The CPU can perform other tasks in the meantime.
 - When the interrupt occurs, the CPU momentarily stops its task, branches to a service program to handle the transfer, and then returns to its original work.
- **Direct Memory Access (DMA):**
 - Used for high-speed data transfer where large blocks of data need to move directly between the device and memory.
 - The **DMA Controller** takes over the system buses to manage the transfer, bypassing the CPU entirely during the actual movement.
 - The CPU only gets involved to set up the transfer and receives an interrupt once the entire block is finished.

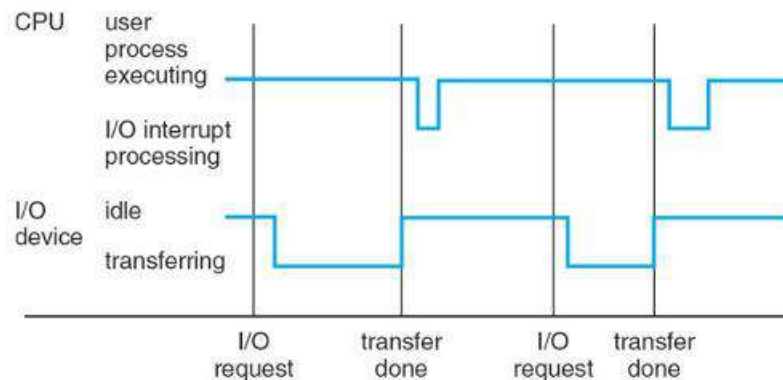


How a Modern Computer Works





Interrupt Timeline



Q3: What are Application Programming Interfaces (APIs)?

An **Application Programming Interface (API)** is a set of functions, rules, and protocols that allows software applications to communicate with the operating system (OS) or other services. APIs provide a simplified way for programmers to access system resources without writing complex low-level code.

In simple terms:

API = what programmers use to request services

System call = what the kernel uses to perform those services

Common Types of APIs

- **Win32 API** – Used in Windows operating systems
- **POSIX API** – Used in UNIX, Linux, and macOS systems
- **Java API** – Used within the Java Virtual Machine (JVM)

Steps Involved in API Implementation

The implementation of a system call through an API follows these steps:

- **The Call:** A user application calls a function provided by the API (e.g., `printf()` in C or `CreateProcess()` in Windows).
- **The Interface:** The “System-Call Interface” intercepts this function call.
- **The Table Search:** The interface looks up the corresponding system call number in a specialized table.
- **Kernel Execution:** The OS switches to kernel mode, executes the specific system call, and performs the requested task.
- **The Return:** Once the task is finished, the OS returns the status or data back to the API, which then passes it to the user program.

Example

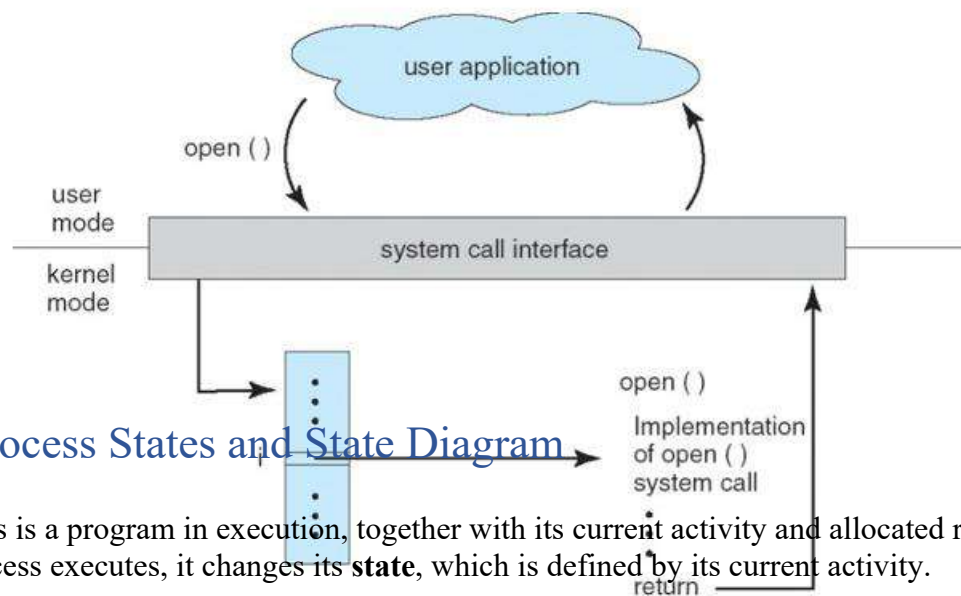
ReadFile() Or PrintFile() (API) → system call → OS kernel → result returned

Parameters:

- **file** – the file to read from
- **buffer** – where the data is stored
- **bytesToRead** – how much data to read
- **bytesRead** – how much data was actually read
- **ovl** – indicates overlapped (asynchronous) I/O
(Simply, ovl tells the system if the program should wait for I/O to finish or do it in the background while continuing.)



API – System Call – OS Relationship



Q4. Process States and State Diagram

A process is a program in execution, together with its current activity and allocated resources. As a process executes, it changes its **state**, which is defined by its current activity.

The Five Process States

- **New:** The process is being created.
- **Ready:** The process is waiting to be assigned to a processor.
- **Running:** Instructions are being executed by the CPU.



- **Waiting:** The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- **Terminated:** The process has finished execution.

State Transitions

- A process moves from **New** to **Ready** once OS **admits** the **new process into memory** and places it in the **ready queue**.

Example: You double-click an app → OS loads it into RAM.

- It moves from **Ready** to **Running** when the scheduler picks it to execute.

Example: Your browser gets CPU time after waiting.

- If its time slice expires or it is interrupted, it moves from **Running** back to **Ready**.

Example: Your music player is paused so a system update can run.

- If a “running” process needs I/O or waits for an event (e.g., file read, user input), it moves to the **Waiting** state.

Example: Word processor waits for you to type something.

- Once I/O or the event is complete, it moves from **Waiting** back to **Ready**.

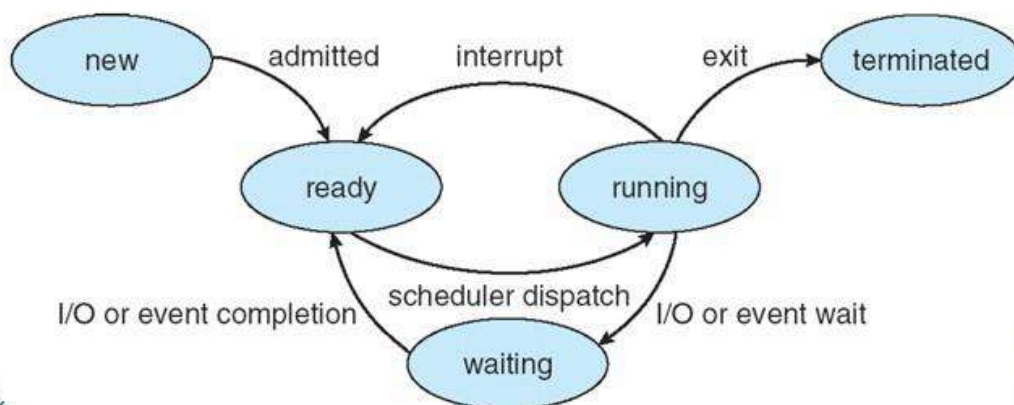
Example: File finishes loading → process is ready to run again.

- Upon completion, it moves to the **Terminated** state.

Example: You close a program or it crashes.

Why This Matters?

- **CPU Utilization:** Keeps CPU busy by switching between processes.
- **Responsiveness:** Lets user interact with programs while they do I/O.
- **Fairness:** No single process hogs the CPU.



Q5 Protection and Security – How OS Manages Them

Protection and **security** are essential functions of an operating system to ensure safe and controlled use of system resources. The Operating System ensures the integrity of the system by distinguishing between these two concepts as:

Protection Management

- **Protection:** A mechanism for controlling access of processes or users to the resources defined by the OS.

The OS uses **Isolation** to ensure that one application cannot interfere with another or access the memory of the kernel using this method:

- Each user is assigned a **User ID (UID)**
- Each file and process is linked with its user ID
- Access permissions are decided based on this ID
- **Group ID (GID)** is used to group users allowing easier management of permissions
- **Privilege escalation** allows a user to temporarily gain higher access rights when needed

Security Management

- **Security:** Defense of the system against internal and external attacks.

The OS implements user permissions (Read/Write/Execute) and uses Dual-Mode Operation to prevent unauthorized access to sensitive hardware or data structures. User authentication verifies identity before access and protect again viruses, Denial-of-service attacks and unauthorized access.

Overall security must be applied throughout the system as a weakness in one part can affect the whole system

Q6. Process Scheduling and Types

The objective of process scheduling is to have some process running at all times to maximize CPU utilization. To achieve this, the Operating System uses different types of schedulers to manage how processes move through the system.

Scheduling Queues

The OS manages processes by placing them into various specialized queues:

- **Job Queue:** The set of all processes in the system.
- **Ready Queue:** The set of all processes residing in main memory, ready and waiting to execute.
- **Device Queues:** A set of processes waiting for a specific I/O device; each device has its own queue.

Types of Schedulers

Operating systems typically utilize three levels of scheduling:

- **Long-Term Scheduler (Job Scheduler):**
 - Selects which processes should be brought into the **ready queue** from the pool of new jobs.
 - It controls the **degree of multiprogramming** (the number of processes in memory).
 - It executes much less frequently than other schedulers.
- **Short-Term Scheduler (CPU Scheduler):**
 - Selects which process from the **ready queue** should be executed next and allocates the CPU.
 - It must be extremely fast because it executes very frequently.
 - In some systems, it is the only scheduler as it decides who gets the CPU.
- **Medium-Term Scheduler:**
 - Used in systems that support **swapping**.
 - It can temporarily remove a process from memory (and from the contention for the CPU) to reduce the degree of multiprogramming and later reintroduce it.

Process Types (I/O-Bound vs. CPU-Bound)

A key responsibility of the long-term scheduler is to maintain a good **process mix** between these two types:

- **I/O-Bound Process:**
 - Spends more of its time doing I/O than doing computations.
 - It is characterized by many short CPU bursts.
- **CPU-Bound Process:**
 - Spends more of its time doing computations.
 - It generates I/O requests infrequently and has very long CPU bursts.

Q7. PCB and Context Switching

Process Control Block (PCB)

Process Control Block (PCB) also called as task control block is a data structure that stores complete information about a process. Without PCB, multitasking is impossible. Each process is represented in the operating system by a PCB. It contains:

- **Process State:** New, ready, running, etc.
- **Program Counter:** Indicates the address of the next instruction to be executed.
- **CPU Registers:** Includes stack pointers, general-purpose registers, etc.
- **CPU Scheduling Information:** Priorities and scheduling queue pointers.
- **Memory-management Information:** Page tables or segment tables.
- **Accounting Information:** CPU used, time limits, account numbers.
- **I/O Status Information:** List of I/O devices allocated and open files.

Context Switching

A context switch occurs when the CPU switches from one process to another.

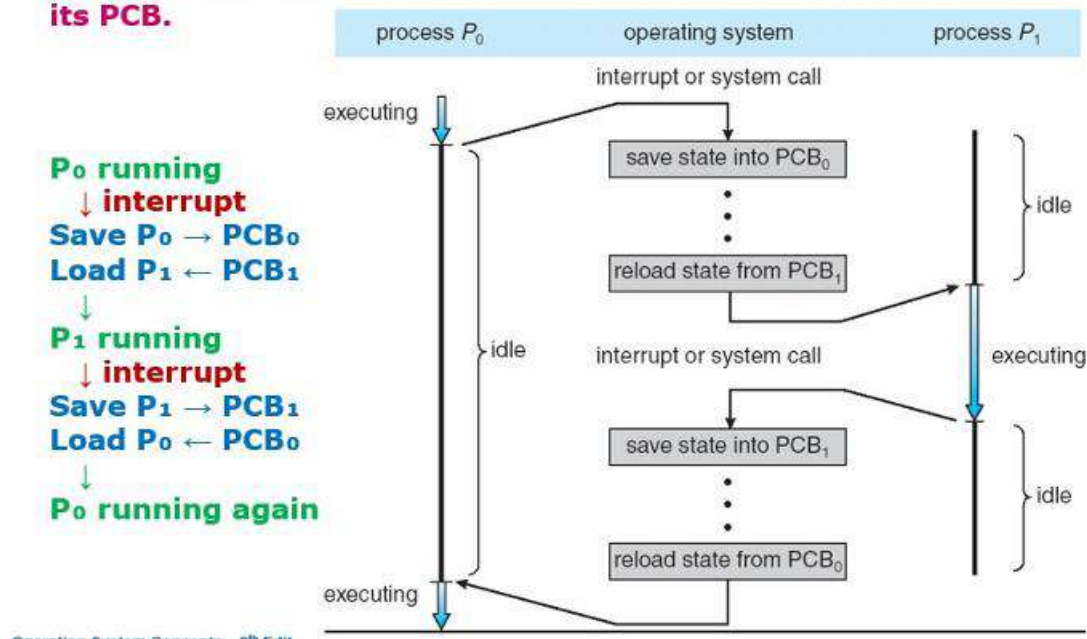
- **Mechanism:** The system saves the state (context) of the current process and loads the saved state for the next process.
- **Storage:** The process state is stored in its respective **PCB**.
- **Overhead:** Context switching is "overhead" because the system does no useful work while switching.
- **Speed factors:** Switch time depends on hardware support (e.g., multiple register setup) and the complexity of the OS.

Steps in a Context Switch:

When a CPU switch occurs:

1. **Process P_0** is interrupted or makes a system call.
2. **Save State:** The OS saves the current state of P_0 into PCB_0
3. **Reload State:** The OS reloads the state of **Process P_1** from PCB_1 by running scheduler.
4. **Execute:** Process P_1 begins or resumes execution.

CPU switch occurs when the OS saves the state of the running process in its PCB and loads the state of another process from its PCB.



Q9: Monolithic and Microkernel System Architecture

1. Monolithic Architecture

Monolithic architecture is an operating system design where the entire OS runs as a single program in kernel mode. All system components such as file system, memory management, and process scheduling are combined into one large binary. Each component can directly communicate with others.

Advantages:

- High performance due to direct communication between components.
- Less overhead since no need for message passing.

Disadvantages:

- Difficult to maintain and extend.
- A failure in one component (e.g., device driver) can crash the entire system.
- Less secure due to lack of isolation.

Examples:

- Traditional UNIX
- Linux
- Windows (largely monolithic with some hybrid features)

2. Microkernel Architecture

Microkernel architecture minimizes the kernel size by keeping only essential functions in kernel mode, while other services run in **user-level programs** (servers). Kernel includes only:

- CPU scheduling
- Memory management
- Inter-process communication (IPC)

Other services (file system, drivers) run as separate user-level processes. Components communicate through message passing via the microkernel.

Advantages:

- More secure and reliable (fault isolation).
- Easier to maintain and extend.
- Better portability across hardware.

Disadvantages:

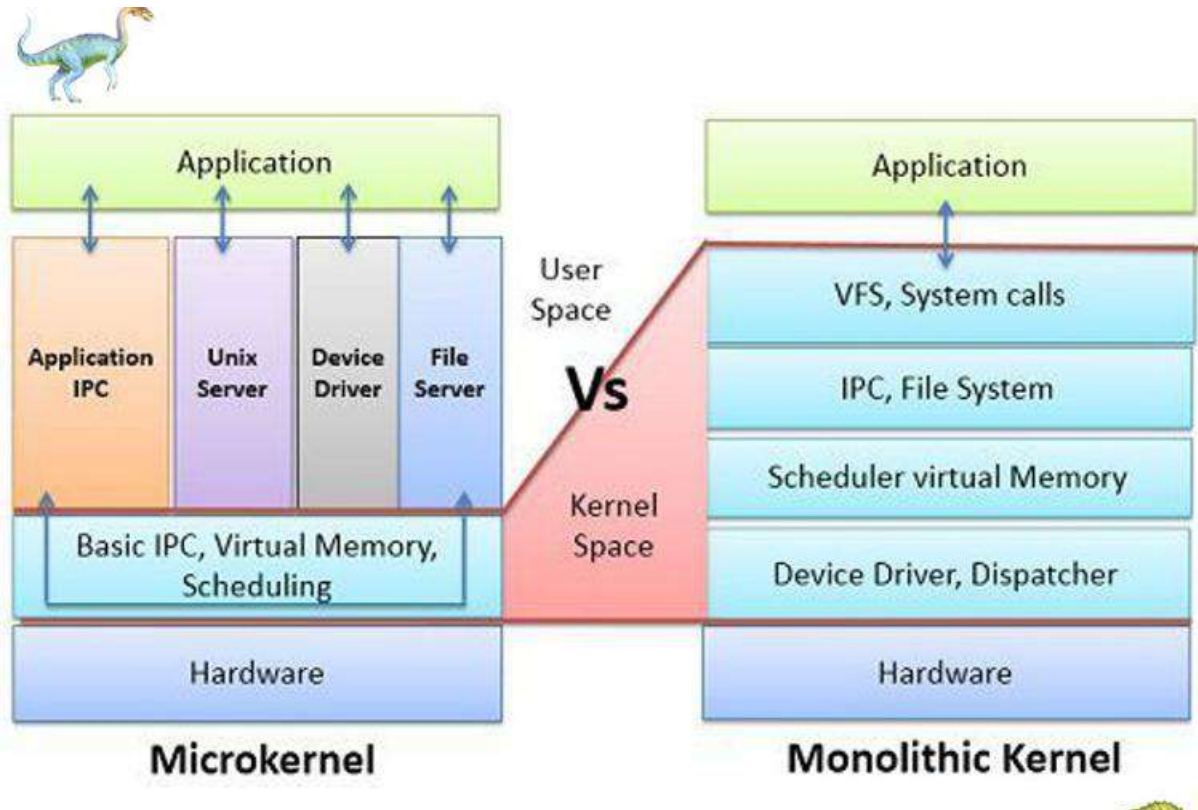
- Lower performance due to:
 - Context switching
 - Message passing overhead

Examples:

- QNX
- Mach (used in macOS)
- L4

Conclusion:

Monolithic systems are faster but less reliable, whereas microkernel systems are more secure and modular but may suffer from performance overhead.



Q10: System Calls (Process Creation and Termination)

System calls provide the interface between a running program and the operating system. They are low-level request made by a program to the OS kernel to perform a task. So, most programmers use APIs' rather than invoking actual system calls directly to maintain portability and ease of use.

Types of System Calls:

- Process control
- File management
- Device management
- Information maintenance
- Communications

- Protection

1. Process Creation

Processes in a system can create multiple new processes via a **create-process** system call.

- **Relationship:** The creating process is the **parent** process, and the new processes are called **children**. These children can, in turn, create other processes, forming a **tree of processes**.
- **Process Identification:** Most operating systems identify processes using a unique **process identifier (pid)**, which is typically an integer.

Resource Sharing Options:

When a child is created, resource sharing can happen in three ways:

1. Parent and children share **all** resources.
2. Children share a **subset** of the parent's resources.
3. Parent and child share **no** resources.

Execution Options:

1. **Concurrent Execution:** The parent and children execute concurrently.
2. **Wait State:** The parent waits until some or all of its children have terminated.

Address Space (Example: UNIX):

- `fork()` system call creates a new process. The new process (child) consists of a **copy** of the address space of the original process (parent).
- `exec()` system call is typically used after a `fork()` to replace the process' memory space with a new program.

2. Process Termination

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the **exit()** system call.

- **Output Data:** The process may return a status value (data) to its parent process via the `wait()` system call.
- **Resource Deallocation:** All resources of the process (physical and virtual memory, open files, I/O buffers) are deallocated by the operating system.

Reasons for Parent Terminating a Child:

A parent may terminate the execution of one of its children (using the `abort()` system call) for reasons such as:

- The child has exceeded its usage of some of the resources it has been allocated.

- The task assigned to the child is no longer required.
- The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.

3. Cascading Termination

Cascading termination is a phenomenon where the operating system does not allow a child process to exist if its parent has been terminated.

- **Mechanism:** If a process terminates (either normally or abnormally), then all its children must also be terminated.
- **The Chain Reaction:** If those children had children of their own, those must also be terminated. This "cascades" down the entire branch of the process tree.
- **Initiation:** This process is normally initiated by the operating system.

Q11: Virtual Machines (Definition and Architecture)

A **Virtual Machine (VM)** is a software implementation of a computer that executes programs like a physical machine. The OS perceives itself as running on its own dedicated hardware, even though it is sharing resources with other VMs.

Importance of Virtual Machines

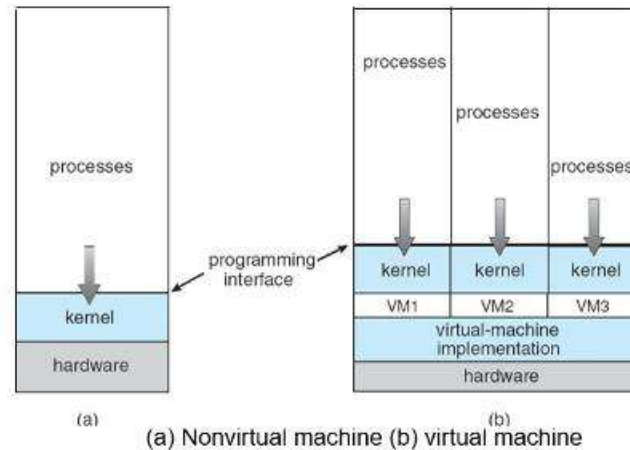
- **Resource Isolation:** VMs provide complete protection of system resources because each virtual machine is isolated from all other virtual machines.
- **Development and Testing:** They allow system programmers to test new operating systems or configurations without crashing the primary "host" hardware.
- **Consolidation:** Multiple different operating systems (e.g., Windows, Linux) can run on the same physical hardware simultaneously.
- **Security:** Since each VM is a separate entity, a virus or failure in one "guest" OS does not affect the host or other guests

Virtual Machine Architecture

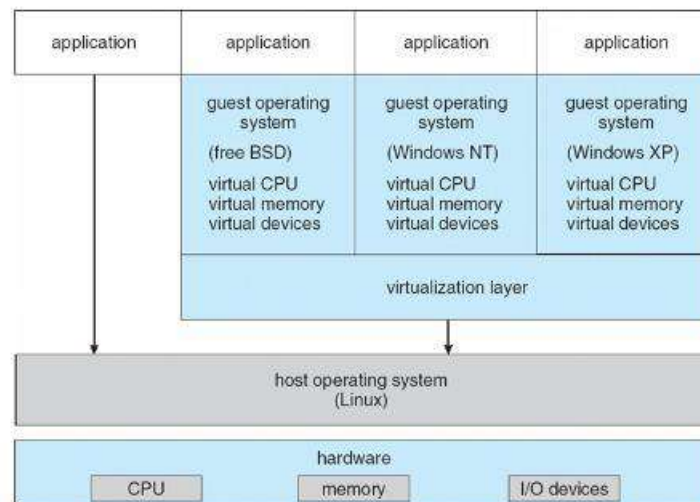
- **Layered Approach:** A virtual machine takes the layered approach to its logical conclusion. It treats hardware and the operating system kernel as though they were all hardware.
- **Host System:** The underlying physical hardware that provides the actual CPU, memory, and disk resources.
- **Virtual Machine Manager (VMM) / Hypervisor:**
 - This is the software layer that creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.
 - It maps the needs of the "guest" operating systems to the physical hardware.
- **Guest OS:** The separate Operating System running inside the VM environment i.e., multiple instances
- **The "Interface":** A virtual machine provides an interface that is identical to the underlying bare hardware.



Virtual Machines (Cont)



VMware Architecture



Q12: Why Users Use APIs

Users and programmers prefer Application Programming Interfaces (APIs) over direct system calls for several important reasons:

- **Program Portability:**

An application written using a standard API can often run on any system that supports that API, regardless of the underlying hardware or specific OS version.

- **Ease of Use:**

Using an API is much easier than handling the complex details of how a system call is actually executed by the kernel.

- **Efficiency:**

APIs provide a high-level, simplified way to perform common tasks (like opening a file or printing text) without needing to manage low-level registers or hardware interrupts manually.

- **Abstraction (Hides Complexity):**
APIs hide the internal workings of the operating system, allowing developers to focus on application logic instead of kernel-level implementation details.
- **Better Maintainability and Reusability:**
Code written using APIs is easier to maintain and update. Developers can reuse API-based code across different projects, saving time and effort.

The API-to-System-Call relationship

Each system call has a unique number internally. The system-call interface maintains a table indexed by these numbers. The API acts as a user-friendly wrapper that invokes the correct system call behind the scenes, completely hiding these details from the programmer.

Q13 IPC Message Passing: Implementation/Linking Issues

When implementing a communication link between processes (P and Q), several design issues must be addressed:

1. Link Establishment

How are links established? They can be established **automatically** by the Operating System when a message is sent, or they can be **manually created** by the programmer.

2. Number of Processes per Link

Can a link connect more than two processes?

- **One-to-one:** Only two specific processes (P and Q) communicate (like a private chat).
- **Many-to-many:** Multiple processes share one link (like a group chat).

3. Number of Links between Processes

How many links can exist between two processes? There might be **just one** path, or **multiple** communication channels (e.g., one for text and another for video).

4. Link Capacity (Buffering)

Capacity refers to how many messages can wait in the queue.

- **Zero Capacity:** No storage; sender must wait for receiver (**Rendezvous**).
- **Bounded Capacity:** Limited size (n messages). Sender waits if the buffer is full.
- **Unbounded Capacity:** Unlimited storage; sender never has to wait.

5. Message Size

- **Fixed Size:** Easier for the OS to manage.

- **Variable Size:** More flexible but harder to manage because each message has a different size.

6. Direction of Communication

- **Unidirectional:** Data flows in one direction only ($P \rightarrow Q$).
- **Bidirectional:** Both processes can send and receive ($P \leftrightarrow Q$).

7. Synchronization (Blocking vs. Non-blocking)

- **Blocking (Synchronous):** The sender or receiver waits until the operation is complete.
- **Non-blocking (Asynchronous):** The sender sends and continues immediately; the receiver gets a message or "null" if none is available.

Implementation of Communication Link

Type	Basis	Examples
Physical	Depends on physical system structure	Shared memory, hardware bus, network
Logical — Direct	Behavior-based: processes explicitly name each other	send(B, msg), receive(A, msg)
Logical — Indirect	Behavior-based: messages go via mailbox/port	send(mailboxA, msg), receive(mailboxA, msg)

Q14 Operating System Operations

OS Operation and Transition (User $\rightarrow$ Kernel Mode)

Modern Operating Systems operate using **Dual-Mode Execution** to protect the system from errant users.

The Two Modes

1. **User Mode (Mode bit = 1):** Executing on behalf of a user application. Access to hardware and memory is restricted.
2. **Kernel Mode (Mode bit = 0):** Executing on behalf of the Operating System. It has unrestricted access to all hardware and privileged instructions.

Transition from User Mode to Kernel Mode

The transition from User Mode to Kernel Mode occurs whenever the system needs to perform a privileged task (like reading from a disk).

1. **User Process:** An application program is running in **User Mode**.
2. **Trap/Interrupt:** The program makes a **System Call** or an interrupt (trap, error) occurs.
3. **Mode Switch:** The hardware switches the **mode bit from 1 to 0**.
4. **Kernel Execution:** The OS takes control in **Kernel Mode**, validates the request, and performs the task.
5. **Return:** Once finished, the OS switches the **mode bit back to 1** and returns control to the user program

Key Features

It prevents user programs from directly accessing critical system resources. Just a system call is used when a user program needs OS services.

Multi-Mode Extension

- Modern CPUs may support additional modes, such as:
 - **Virtual Machine Manager (VMM) mode** for virtual machines

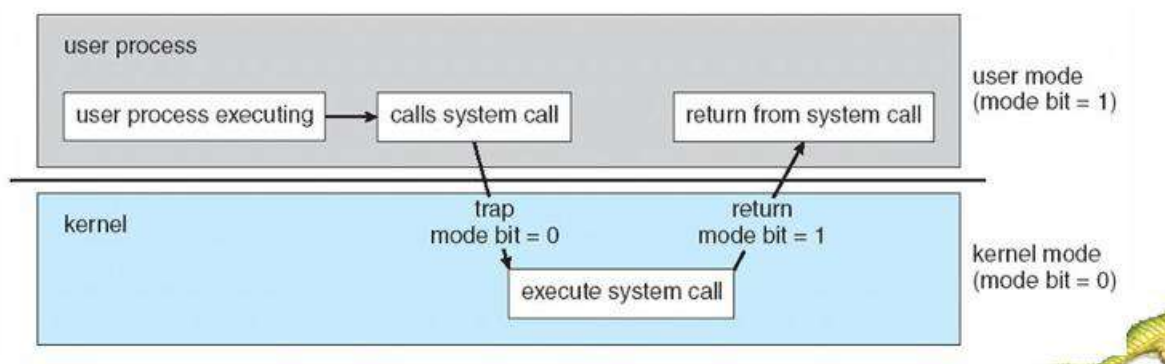
Role of Timer in Protection

To prevent misuse of CPU (e.g., infinite loops), a **timer mechanism** is used:

- A timer is set to generate an interrupt after a fixed time
- A **counter** is decremented by the system clock
- The OS sets this counter using a **privileged instruction**
- When the counter reaches zero:
 - An interrupt is generated
 - The OS regains control

Purpose

- Prevents processes from running indefinitely
- Ensures fair CPU sharing
- Allows OS to terminate misbehaving programs



1. Why does FIFO suffer from Belady's Anomaly?

Belady's Anomaly is a phenomenon in which increasing the number of page frames results in an increase in the number of page faults, which is contrary to expectations.

FIFO (First-In, First-Out) suffers from this anomaly because:

- It always removes the oldest page in memory, regardless of how frequently or recently it has been used.
- It does not consider page usage patterns when making replacement decisions.
- A heavily used page may be removed simply because it has been in memory the longest.

Therefore, FIFO makes replacement decisions without considering page importance, which can sometimes lead to poorer performance even when more memory is available.

2. Why is the Optimal Page Replacement Algorithm not used in real systems?

It is not used in real systems because:

- It requires exact knowledge of future memory references.
- Operating systems cannot accurately predict future page accesses.

As a result, the Optimal algorithm is not practically implementable and is used mainly as a theoretical benchmark for evaluating other page replacement algorithms.

3. When might MRU perform better than LRU?

MRU (Most Recently Used) replaces the page that was accessed most recently.

MRU can outperform LRU (Least Recently Used) in situations where recently accessed pages are unlikely to be used again soon.

A common example is **sequential access**:

- When reading a file from beginning to end, pages that have just been accessed are often not needed again immediately.
- Removing the most recently used page may therefore be more efficient than removing older pages.

In such workloads, MRU can achieve better performance because recent access does not necessarily indicate future use.

Summary

- **Optimal** generally provides the best performance because it minimizes page faults.
- **LRU** is a practical approximation of Optimal and usually performs well.
- **FIFO** is simple to implement but can suffer from Belady's Anomaly.
- **MRU** may outperform LRU in specific access patterns, such as sequential data processing.

Final PrepMaterial

Turnaround Time = Completion Time – Arrival Time

Waiting Time = Turnaround Time – Burst Time

1. When to use the formula (Calculate it yourself)

If the problem states the **total maximum resources** that exist in the entire system, you *must* use the formula:

Current Available = Total Resources – Sum of Allocated

2. When NOT to use the formula (Use the given numbers directly)

If the problem explicitly states what is **currently sitting in the free pool**, you do *not* subtract anything. The subtraction was already done for you by whoever wrote the question.

- **Your previous image did this.**
 - **The phrasing:** "There are 3 units of type X, 2 units of type Y and 2 units of type Z **still available.**"
-

Q1. CPU Scheduling Algorithms & Gantt Charts

CPU scheduling determines which process runs on the CPU when multiple processes are in the ready queue. The scheduler selects a process based on a particular algorithm. Key metrics used to evaluate algorithms are:

Metric	Definition
Waiting Time (WT)	Time a process spends in the ready queue waiting for CPU.
Turnaround Time (TAT)	Total time from submission to completion. $TAT = Completion\ Time - Arrival\ Time$
Average WT	Sum of all waiting times $\div$ number of processes
Average TAT	Sum of all turnaround times $\div$ number of processes

Given processes (as per provided solution):

Process	Burst Time	Priority
P1	8	4
P2	6	1
P3	9	2
P4	3	3
P5	1	2

1. First Come First Served (FCFS)

- Processes are executed in the order they arrive in the ready queue.
- FCFS is a non-preemptive scheduling algorithm i.e. once a process starts execution, it runs until completion.
- It is simple to implement but can cause convoy effect (short processes wait behind long ones).
- Convoy Effect: A long process holds the CPU; all short processes queue behind it resulting in poor average WT reducing overall system performance.

Execution Order: P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P5 (assumed arrival time = 0)

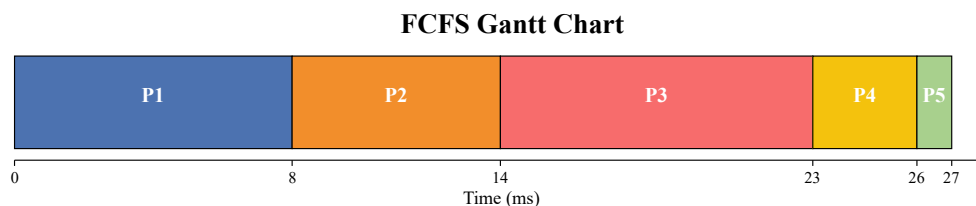
Process	Start	Finish	WT (TAT – BT)	TAT (CT – AT)
P1	0	8	0	8
P2	8	14	8	14
P3	14	23	14	23
P4	23	26	23	26

Process	Start	Finish	WT (TAT – BT)	TAT (CT – AT)
P5	26	27	26	27

Avg. Wait = $(0+8+14+23+26)/5 = 71/5 = 14.2$ ms

Avg. TAT = $(8 + 14 + 23 + 26 + 27) / 5 = 98/5 = 19.6$ ms

FCFS Gantt Chart



2. Shortest Job First (SJF)

- The process with the shortest CPU burst time is selected for next execution.
- It is optimal as it gives minimum average waiting time among all scheduling algorithms.
- However its main disadvantage is that the CPU burst time must be known or estimated in advance (not always possible).

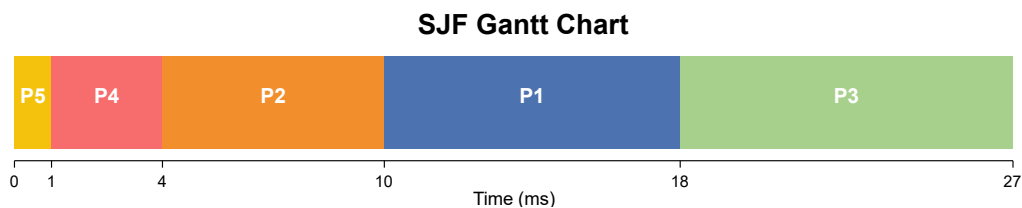
Order by burst: P5(1) → P4(3) → P2(6) → P1(8) → P3(9)

Process	Burst	Start	Finish	WT	TAT
P5	1	0	1	0	1
P4	3	1	4	1	4
P2	6	4	10	4	10
P1	8	10	18	10	18
P3	9	18	27	18	27

Avg. Wait = $(0+1+4+10+18)/5 = 33/5 = 6.6$ ms

Avg. TAT = $(1+4+10+18+27)/5 = 60/5 = 12$ ms

SJF Gantt Chart



3. Priority Scheduling

- Each process is assigned a priority value, and the CPU is allocated to the process with the highest priority. By convention, a lower priority value indicates a higher priority.

- A major disadvantage is starvation, where low-priority processes may wait indefinitely.
- So, Aging is used to prevent starvation by gradually increasing the priority of waiting processes.

Order by priority (lowest priority = highest): P2(1) → P3(2) → P5(2) → P4(3) → P1(4)

Process	Priority	Start	Finish	WT	TAT
P2	1	0	6	0	6
P3	2	6	15	6	15
P5	2	15	16	15	16
P4	3	16	19	16	19
P1	4	19	27	19	27

Avg. Wait = $(0+6+15+16+19)/5 = 56/5 = 11.2$ ms

Avg. TAT = $(6+15+16+19+27)/5 = 83/5 = 16.6$ ms

Non-Preemptive Priority Gantt Chart

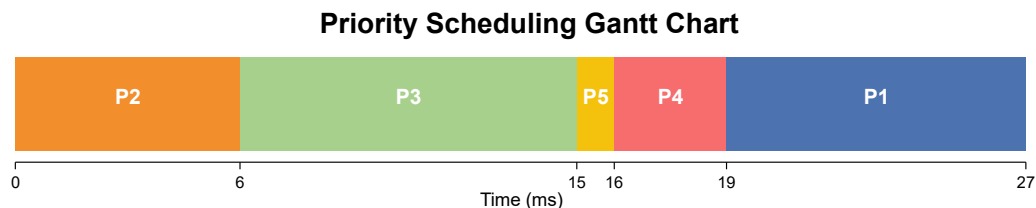


Figure 3: Priority — lower number = higher priority

4. Round Robin (RR)

- Each process is assigned a fixed time slice called the **time quantum (q)**.
- Round Robin is a preemptive scheduling algorithm. After q expires, process is preempted and moved to end of ready circular queue.
- It is commonly used in time-sharing systems because it provides fairness to all processes.
- If q is large → behaves like FCFS. If q is small → can cause high context switch overhead.

Round Robin (q = 1 ms)

All processes take turns one millisecond at a time in order: P1, P2, P3, P4, P5, P1, P2...

Find Completion Times and TAT

Look for the **last occurrence** of each process.

Since all arrival times = 0:

$$\text{TAT} = \text{CT} - \text{AT} = \text{CT}$$

Processes	CT	TAT
P1	25	25

P2	21	21
P3	27	27
P4	13	13
P5	5	5

Calculate WT

$$\text{WT} = \text{TAT} - \text{Burst Time}$$

Process	TAT	Burst	WT
P1	25	8	17
P2	21	6	15
P3	27	9	18
P4	13	3	10
P5	5	1	4

Waiting Time Calculation:

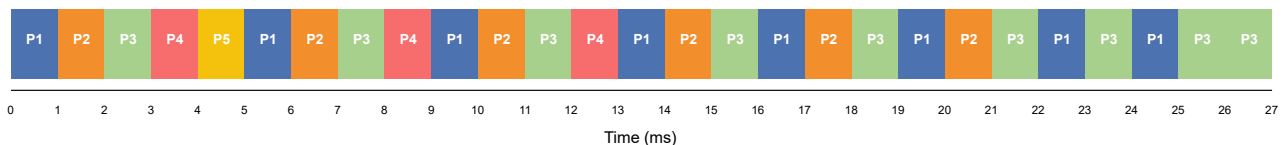
T1 Runs at: 0, 5, 9, 13, 16, 19, 22, 24

- P1 WT = 0 + 4 + 3 + 3 + 2 + 2 + 2 + 1 = 17
- P2 WT = 1 + 4 + 3 + 3 + 2 + 2 = 15
- P3 WT = 2 + 4 + 3 + 3 + 2 + 2 + 1 + 1 = 18
- P4 WT = 3 + 4 + 3 = 10
- P5 WT = 4

Avg. WT = (17 + 15 + 18 + 10 + 4) / 5 = 60 / 5 = 12.8 ms

Avg. TAT = (25 + 21 + 27 + 13 + 5) / 5 = 87 / 5 = 18.2 ms

Round Robin (q = 1) Gantt Chart



Algorithm Comparison Summary

Algorithm	Avg. Wait Time	Avg. TAT	Notes
FCFS	14.2 ms	19.6 ms	Simple, may suffer from convoy effect
SJF	6.6 ms	12.0 ms	Minimum average waiting and turnaround time
Priority (Non-Preemptive)	11.2 ms	16.6 ms	May cause starvation
Round Robin (q = 1 ms)	12.8 ms	18.2 ms	Fair scheduling, higher context-switch overhead

SJF gives minimum average waiting time and minimum average TAT among all algorithms.

Q2. Computer Architecture & OS Structure

Part (a): How the Computer Works — Von Neumann Architecture

The **Von Neumann architecture** is the standard design model used in most modern computers. It defines how computer hardware components are organized and how instructions are executed. Its main principle is the **stored-program concept**, where both program instructions and data are stored in the same memory.

The main components of the Von Neumann architecture are:

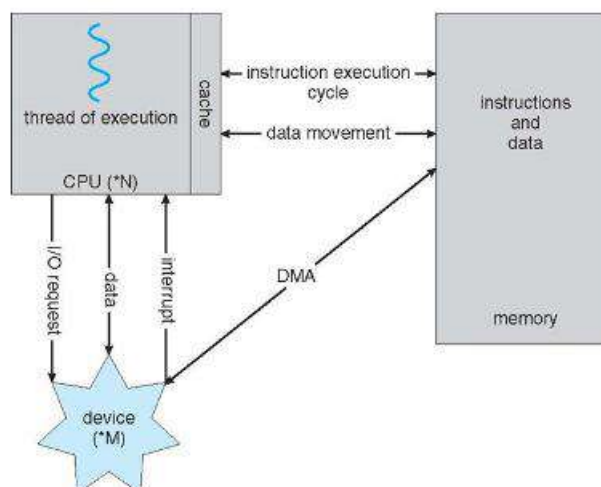
- **CPU (Central Processing Unit):** Executes instructions and controls computer operations.
- **ALU (Arithmetic Logic Unit):** Performs arithmetic and logical operations.
- **Control Unit (CU):** Fetches, decodes, and controls the execution of instructions.
- **Memory (RAM):** Stores data and program instructions.
- **Input/Output (I/O) Devices:** Allow communication between the computer and external devices.
- **System Bus:** Connects the CPU, memory, and I/O devices and transfers data, addresses, and control signals.

Working Principle

1. When the computer is powered on, the bootstrap program stored in ROM starts executing.
2. The bootstrap program initializes the hardware and loads the operating system into RAM.
3. The operating system takes control of the system and manages the CPU, memory, and I/O devices.
4. When a user runs a program, the program and its data are loaded into main memory (RAM).
5. The CPU repeatedly performs the Fetch–Decode–Execute cycle for each instruction:
 - The Control Unit fetches an instruction from memory.
 - The instruction is decoded to determine the required operation.
 - The ALU or another CPU component executes the instruction.
 - The result is stored in a register or memory.
 - The Program Counter (PC) is updated to point to the next instruction.
6. I/O devices communicate with the CPU through device controllers and interrupts are used to notify the CPU when an I/O operation requires attention or has completed.



How a Modern Computer Works



FETCH → DECODE → EXECUTE → STORE (repeat)

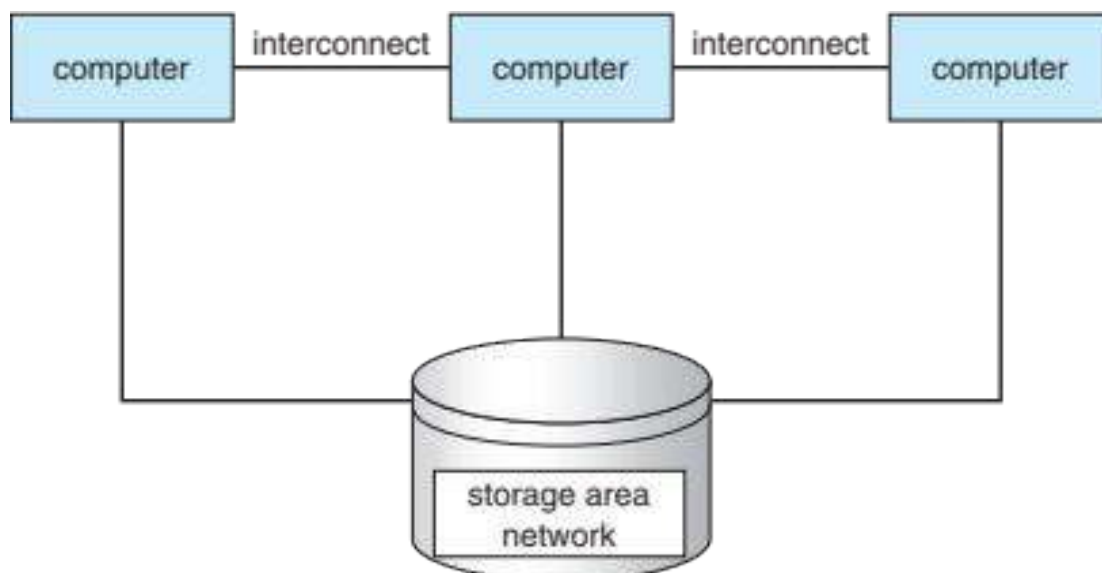
Part (b): System Differences

1. Asymmetric vs Symmetric Clustering

Feature	Asymmetric Clustering	Symmetric Clustering
Definition	One node actively runs the service while another remains in standby mode ready to take over if a failure occurs.	All nodes are active and can run applications while monitoring each other.
Active Nodes	It has one active node and other standby idle nodes.	All nodes are active simultaneously.
Fault Tolerance	The standby node takes over if active node fails.	Any node can take over the workload of a failed node.
Resource Use	It has lower efficiency because standby nodes are mostly idle.	It has higher efficiency because all nodes contribute resources.
Performance	Focuses mainly on high availability rather than performance improvement.	Provides both high availability and improved performance through load sharing.
Example	Used in critical services requiring failover protection, such as database servers with hot-standby failover	Used in Load balancing, web services, and parallel processing environments.

Asymmetric Clustering Diagram

For symmetric remove interconnections with all active while here 1st one is active with 2nd and 3rd being standby.



2. Monolithic vs Microkernel OS Structure

Feature	Monolithic (UNIX-style)	Microkernel
Definition	All OS services in one large kernel block.	Only essential services in kernel; rest in user space.
What's in Kernel	File system, device drivers, memory, scheduling, system calls.	CPU scheduling, memory management, IPC only.
What's in User Space	Almost nothing (runs in kernel mode).	File system, device drivers, network services.
Communication	Direct function calls within kernel.	Message passing between user-space modules.
Performance	Fast (direct calls, no message overhead).	Slower (message passing overhead).
Stability	One bug can crash entire system.	Module crashes doesn't crash kernel.
Example	Traditional UNIX, Linux.	Mac OS X (Darwin), Minix, QNX.

3. Single vs Multi Processing

Feature	Single Processing	Multi Processing
CPUs	One CPU or core executes all instructions.	Multiple CPUs/cores execute processes simulatenously.
Parallelism	No true parallel execution; tasks are handled one at a time using time-sharing (context switching).	True parallel execution of multiple processes at same time.
Throughput	Throughput is limited by single CPU speed.	Higher throughput due to simultaneous execution of processes.
Scalability	Limited scalability; performance improvements mainly require a faster CPU.	Highly scalable; performance can be increased by adding more CPUs or cores.
Reliability	If the CPU fails, the entire system stops working.	Provides graceful degradation, system continues even if one CPU fails.
Cost	It has lower cost and simpler design.	Higher cost but better performance and resource utilization.
Types	It can not be classified further.	Can be classified into Asymmetric Multiprocessing (AMP) and Symmetric Multiprocessing (SMP).

4. Asymmetric vs Symmetric Multiprocessing

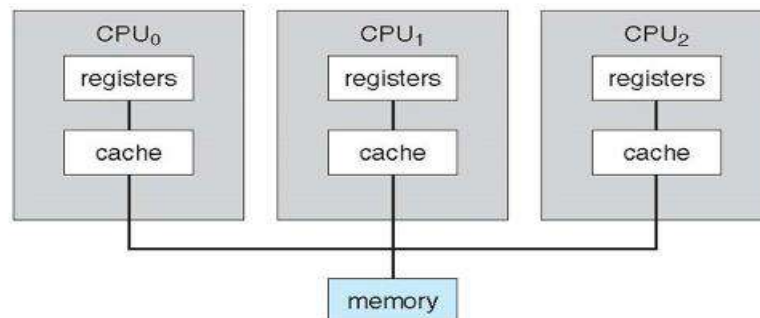
Feature	Asymmetric Multiprocessing (AMP)	Symmetric Multiprocessing (SMP)
Definition	Each processor is assigned a specific task, and one master processor usually controls the system.	All processors are equal and share control of the system.
Control	Master-slave relationship exists.	No master processor; all processors are peers.
Task Assignment	The master processor assigns work to the slave processors.	The operating system schedules tasks on any available processor.
Resource Access	The master manages access to memory and I/O resources.	All processors share memory and I/O resources equally.
Reliability	Failure of the master processor may cause system failure.	More fault-tolerant because no single processor controls the system.
Performance	It is less efficient due to centralized control and possible bottlenecks.	More efficient because workload is distributed among processors.
Complexity	It is simpler to design and manage.	More complex due to synchronization and shared-resource management.
Example	Early multiprocessor systems and some embedded systems.	Modern multicore PCs, workstations, and servers.

Symmetric Multiprocessing (SMP) Architecture Diagram

For Asymmetric just make a MASTER-CPU as head (runs OS/kernel)

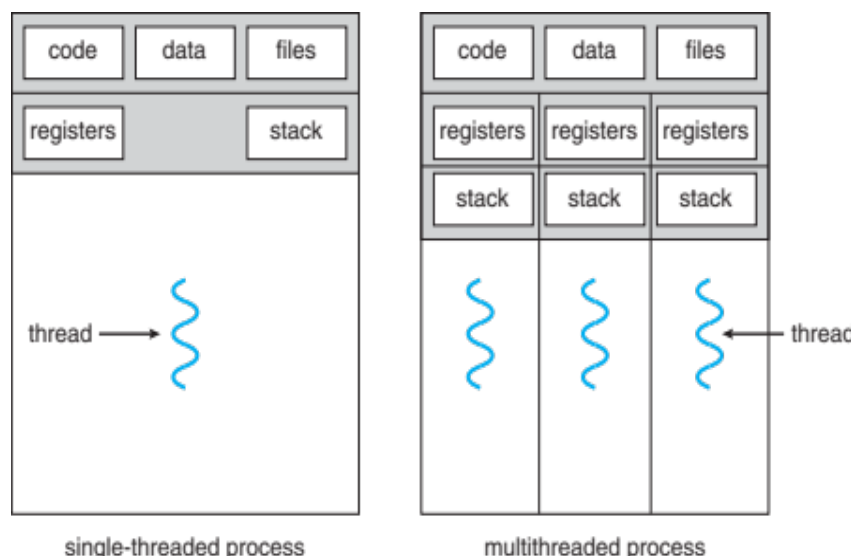


Symmetric Multiprocessing Architecture



Difference Between Single-Threaded and Multithreaded Processes

Single-Threaded Process	Multithreaded Process
Contains only one thread of execution .	Contains two or more threads of execution .
Executes one task at a time.	Can execute multiple tasks concurrently.
Simpler to design and manage.	More complex due to thread synchronization and communication.
If the thread blocks, the entire process stops until the operation completes.	If one thread blocks, other threads can continue running.
Lower overhead because only one thread is maintained.	Higher overhead due to management of multiple threads.
Less responsive for interactive applications.	More responsive, especially for user interfaces and servers.
Example: A simple calculator program.	Example: A web browser where different tabs run simultaneously.



Q3. Banker's Algorithm — Deadlock Avoidance

Part (a): Banker's Algorithm — Full Explanation & Safe State Check

The Banker's Algorithm is a deadlock avoidance algorithm used by the OS to decide whether a resource request should be granted. It only allocates resources if the system remains in a SAFE STATE after allocation.

Key Data Structures

Data Structure	Meaning
Allocation[i]	Resources currently allocated to process P_i .
Max[i]	Maximum resources that process P_i may ever need.
Need[i]	Resources still needed.

Data Structure	Meaning
	Formula: $\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$
Available	Currently free/unallocated resources in the system.
Work	Working copy of Available used during safety check.
Finish[i]	Boolean: has process i finished? Initially false for all.

Safe State Definition

- A system is in a SAFE STATE if there exists a SAFE SEQUENCE i.e., an order in which all processes can finish.
- In the safe sequence, each process can get resources it needs from currently available + resources held by processes that have already finished.
- Safe State means no deadlock WILL happen.
- Unsafe State: no guaranteed safe sequence exists, deadlock may occur.

Safety Algorithm — Step by Step

1. Initialize $\text{Work} = \text{Available}$ and set $\text{Finish}[i] = \text{false}$ for all processes.
2. Find a process P_i such that $\text{Finish}[i] = \text{false}$ and $\text{Need}[i] \leq \text{Work}$.
3. If found, set $\text{Work} = \text{Work} + \text{Allocation}[i]$ and mark $\text{Finish}[i] = \text{true}$.
4. Repeat Step 2 until no such process is found.
5. If all $\text{Finish}[i] = \text{true} \rightarrow \text{SAFE STATE}$, otherwise $\rightarrow \text{UNSAFE STATE}$.

Initialize $\rightarrow$ Find $\rightarrow$ Execute $\rightarrow$ Repeat $\rightarrow$ Check

1. When to use the formula (Calculate it yourself)

If the problem states the **total maximum resources** that exist in the entire system, you *must* use the formula:

Current Available = Total Resources – Sum of Allocated

2. When NOT to use the formula (Use the given numbers directly)

If the problem explicitly states what is **currently sitting in the free pool**, you do *not* subtract anything. The subtraction was already done for you by whoever wrote the question.

- **Your previous image did this.**
- **The phrasing:** "There are 3 units of type X, 2 units of type Y and 2 units of type Z **still available**."

Complete Worked Example

In slide 15

Part (b): Resource Request Algorithm — Grant or Deny?

When process P_i makes a request $Request[i]$ for resources, the OS follows this algorithm BEFORE granting:

Algorithm Steps

STEP 1: Check $Request \leq Need$

- If $Request[i] \leq Need[i] \rightarrow$ proceed to Step 2
- Else $\rightarrow$ ERROR: process has exceeded its maximum claim. Deny request.

STEP 2: Check $Request \leq Available$

- If $Request[i] \leq Available \rightarrow$ proceed to Step 3
- Else $\rightarrow$ process must WAIT (resources not currently available).

STEP 3: Tentatively Allocate Resources (Pretend to Grant)

- $Available = Available - Request[i]$ (subtract from free pool)
- $Allocation[i] = Allocation[i] + Request[i]$ (add to P_i 's allocation)
- $Need[i] = Need[i] - Request[i]$ (P_i needs less now)

STEP 4: Run Safety Algorithm on New State

- Run the full safety algorithm (as in Part a) with the new Available, Allocation, Need values.
- If SAFE STATE found $\rightarrow$ GRANT the request. Allocation is permanent.
- If UNSAFE STATE $\rightarrow$ ROLLBACK. Restore old Available, Allocation, Need. Process MUST WAIT.

Part B: Resource Request Algorithm — Granting or Denying a Request

Summary

Condition Fails?	Action
$Request > Need[i]$	DENY — process exceeded maximum declared need (error).
$Request > Available$	WAIT — not enough resources free right now.
Safety check $\rightarrow$ Unsafe	ROLLBACK & WAIT — granting would risk deadlock.
All checks pass + Safe	GRANT — allocation is safe.

Q4. Threads, Readers-Writer Problem & Critical Section

Part (a): Threads — Core Concepts

What is a Thread?

A thread is the smallest unit of execution within a process. A single process can contain multiple threads, all sharing the same memory space, code, and resources.

Thread vs Process	Thread	Process
Memory	Shares memory with other threads in same process.	Has its own separate memory space.
Creation Cost	Lightweight and faster to create.	Heavyweight and slower to create.
Communication	Direct communication through shared memory, no OS needed	Requires Inter-Process Communication (IPC) mechanisms (message passing, pipes, shared memory with OS help).
Failure	One thread failure can affect the entire process.	Process failure is isolated from other processes.

Benefits of Threads

- **Responsiveness:** If one thread is blocked (e.g., I/O), other threads continue execution, preventing the application from freezing.
- **Resource Sharing:** Threads share memory, so communication is fast and does not require IPC overhead.
- **Economy:** Thread creation and switching are cheaper than process creation.
- **Scalability:** On multicore systems, threads can execute in parallel on different cores.

task_struct — Linux Process/Thread Structure

In Linux, both processes and threads are represented using the `task_struct` structure (also called the Process Control Block, PCB). It contains:

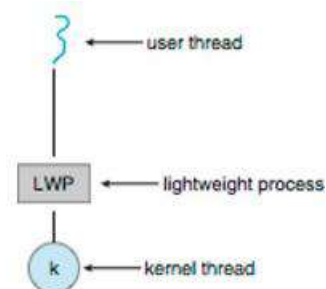
- Process ID (PID), state, and priority
- Program counter and CPU registers
- Memory management information (code, data, heap, stack)
- Open file descriptors and signals information
- Scheduling information

Threads in Linux are created using the `clone()` system call, which allows sharing of resources like memory and files. Therefore, threads are treated as lightweight processes with shared `task_struct` resources.

Lightweight Process (LWP)

A Lightweight Process (LWP) is an intermediate structure between a user-level thread and a kernel-level process. It acts as a virtual CPU for user threads.

Each LWP is mapped to a kernel thread. User-level threads are scheduled onto LWPs by a thread library. If one LWP becomes blocked (for example, while performing an I/O operation), only that LWP and the user threads running on it are blocked, other LWPs can continue executing independently.



OpenMP Threads (Implicit Threading)

OpenMP is a compiler-based API for shared-memory parallel programming in C/C++/Fortran. It provides **implicit threading**, where the programmer uses compiler directives such as `#pragma omp parallel`, and the OpenMP runtime system automatically manages thread creation, scheduling, and synchronization.

- The **OpenMP runtime system** determines the number of threads, based on environment settings (e.g., `OMP_NUM_THREADS`) and available hardware resources.
- The programmer writes mostly sequential code but explicitly marks regions to be parallelized. OpenMP does not automatically parallelize all code.
- The programmer does not manually manage, create or join threads; the runtime handles thread teams internally.
- Example: `#pragma omp parallel` distributes for-loop iterations among threads in a team.

Example:

OpenMP – mark loops as parallel, compiler handles threads.

- Programmer just marks which loops or code blocks can run in parallel.
- Compiler automatically makes threads.

```
#pragma omp parallel for
for(i=0; i<N; i++) {
    c[i] = a[i] + b[i];
}
```

Without OpenMP

(normal loop):

i=0 -> c[0]=6

i=1 -> c[1]=8

i=2 -> c[2]=10

i=3 -> c[3]=12

With OpenMP, the system can do this at the same time:

•Core 1: i=0 -> c[0]=6

•Core 2: i=1 -> c[1]=8

•Core 3: i=2 -> c[2]=10

•Core 4: i=3 -> c[3]=12

Result is the same, but loop finishes faster because multiple cores are working together.

• `#pragma omp parallel for`: "This for loop can run in parallel on multiple CPU cores."

• You don't create threads manually; OpenMP does it.

• Each loop iteration (i) is **independent** and runs as a **separate task**.

• OpenMP automatically **assigns iterations to available cores**.



Key Definitions

Term	Definition
Parallelism	Execution of multiple tasks simultaneously on multiple CPU cores
Concurrency	Multiple tasks making progress at the same time, possibly using time-sharing on a single core.
Data Parallelism	Same operation applied to different portions of data on multiple cores in parallel.
Task Parallelism	Different operations (tasks) executed on different cores simultaneously.
Amdahl's Law	Speedup is limited by the serial portion of a program: $\text{Speedup} = 1 / (S + (1 - S) / N)$ More serial code → less benefit from extra cores.
Thread Pool	A collection of pre-created threads that are reused to execute multiple tasks. Reusing threads

Term	Definition
	is faster than creating new ones each time.
Context Switch	A process in which OS saves the state of the current process or thread and restores the state of another, introducing overhead without performing useful computation.
Priority Inversion	A situation when a high-priority thread is indirectly delayed by a lower-priority thread holding a required resource.

Part (b): Readers-Writer Problem

The Readers–Writer problem is a classic synchronization problem in which a shared resource (such as a file or database) is accessed by two types of processes: Readers (can share) and Writers (need exclusive access).

Rules

- Multiple readers can read simultaneously.
- Only ONE writer can write at a time.
- When a writer is writing, no reader and no other writer is allowed to access the resource.
- A writer must wait if even ONE reader is reading (Readers are allowed concurrent access as long as no writer is active.)

A **semaphore (Locking mechanism)** is a variable used to control access to shared resources.

Variables Used

Variable	Initial Value	Purpose
x (semaphore)	1	Protects the readercount variable by ensuring mutual exclusion during updates.
y (semaphore)	1	Provides controls access to shared resource i.e., blocks writers when readers are inside.
readercount	0	Tracks number of active readers currently reading.

Reader Algorithm

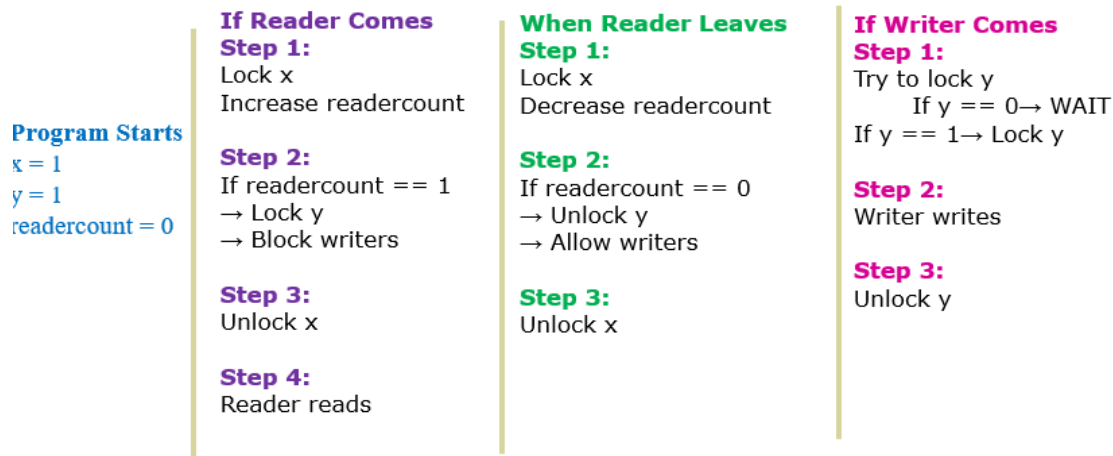
Program starts with $x = 1$, $y = 1$ and $\text{readercount} = 0$

- Wait (lock) x to update readercount safely.
- Increment readercount.
- If $\text{readercount} == 1$, wait (lock) y to block writers.
- Signal (unlock) x and perform reading.
- Wait (lock) x again.
- Decrement readercount.

- If readercount == 0, signal (unlock) y to allow writers.
- Signal (unlock) x.

Writer Algorithm

- Wait (lock) y to get exclusive access to the resource.
- Perform writing.
- Signal (unlock) y.



Full Scenario Trace

From slide 14 OR pdf.

Condition Summary

Condition	Meaning
Resource Busy Condition: y == 0	Resource is currently being used by either a writer or readers (protected state).
Readers Active Condition readercount > 0	One or more readers are currently reading.
Resource Free Condition y == 1 and readercount == 0	Resource is free; no readers or writers are active.
Reader Waiting Condition y == 0 AND readercount == 0	Reader waits with writer writing.
x == 0	A process is currently updating readercount (temporary mutual exclusion).

Dining-Philosophers Problem

The Dining-Philosophers Problem is a classic synchronization problem in OS. It illustrates the challenges of allocating limited resources among multiple processes without causing deadlock or starvation.

In the problem, five philosophers sit around a table. Each philosopher alternates between thinking and eating. There is one fork between each pair of philosophers, so each philosopher needs two forks (left and right) to eat.

The main issue is that if all philosophers pick up one fork at the same time, they may wait indefinitely for the second fork, leading to **deadlock**. The problem is used to study process synchronization, resource sharing and deadlock prevention techniques.

Part (c): Critical Section Problem & OS Solution

What is the Critical Section?

In a system with multiple concurrent processes, the critical section is the section of code where a process accesses or modifies shared resources (variables, files, data structures).

Problem arises here if two or more processes execute their critical sections simultaneously, it leads to a **race condition** causing data inconsistency. Therefore, a proper protocol is needed to control access.

Structure of a Process

Section	Role
Entry Section	Requests permission to enter the critical section.
Critical Section	Accesses or modifies shared resources.
Exit Section	Releases the critical section after execution.
Remainder Section	Performs non-critical operations. (Rest of the process code)

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Requirements for a Valid Solution

A valid solution to the critical section problem must satisfy:

- **Mutual Exclusion:** If process P_i is in its critical section then NO other process can be in their critical section simultaneously.
- **Progress:** If no process is in the critical section and some processes want to enter, the selection of the next process cannot be delayed indefinitely.
- **Bounded Waiting:** After a process requests entry, there must be a limit on how many times other processes can enter before it is granted access (prevents starvation for fair turn).

How OS Handles Critical Sections

OS uses two types of kernels design and synchronization mechanisms to manage critical sections safely:

- **Non-Preemptive Kernel:**
Once a process enters kernel mode, it runs until it voluntarily exits or blocks. Since no interruption occurs in kernel mode, race conditions inside the kernel are avoided. However, this reduces responsiveness.

Example:

- Process A is updating a file in kernel mode.
- It must finish the update fully.
- Process B waits outside until A is done.

- **Preemptive Kernel:**

A running process can be interrupted even in kernel mode. This improves responsiveness but requires synchronization mechanisms to prevent race conditions.

Example:

- Process A is updating a file.
- OS stops A in the middle.
- Process B starts updating the same file.

Modern operating systems primarily use **preemptive kernels** which require proper synchronization techniques. These include mutex locks for exclusive access to critical sections, semaphores for controlling access to shared resources, spinlocks for short waiting periods in multiprocessor systems, and read-write locks. Together, these mechanisms ensure safe coordination between processes and prevent race conditions in concurrent execution.

Peterson's Solution (Software Solution)

Peterson's Solution is a classical software-based method for solving the critical section problem for two processes without hardware support. It uses:

- A shared variable indicating whether a process wants to enter its critical section.
- A shared variable (`turn`) that decide which process gets priority when both processes want to enter simultaneously.

Limitation: It works only for **two processes** and relies on busy waiting, making it impractical for modern operating systems. Modern OSs use synchronization mechanisms such as mutexes, semaphores, spinlocks, and read-write locks instead.

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while (true);
```

Q5. Linux vs Windows Threads & POSIX/Implicit Threading

Part (a): Linux Threads vs Windows Threads

Linux Thread Implementation

In Linux, there is no strict difference between a process and a thread at the kernel level. Both are treated as tasks and represented using a single structure called **task_struct**.

Threads in Linux are created using the `clone()` system call, which allows control over resource sharing between parent and child tasks. Unlike `fork()`, which creates a separate process, `clone()` can create lightweight threads by sharing resources.

The `clone()` system call uses flags to determine what is shared:

- **CLONE_VM**: Shares the same memory space (address space)
- **CLONE_FS**: Shares file system information
- **CLONE_FILES**: Shares open file descriptors
- **CLONE_SIGHAND**: Shares signal handlers

A Linux thread is therefore a task that shares most resources with other threads of the same process. Each thread has:

- Its own **program counter**
- Its own **register set**
- Its own **kernel stack**
- Shared memory and resources (depending on `clone()` flags)

The POSIX thread library (**pthread**s) is used in user space, and internally it maps to `clone()` system calls.

Windows Thread Implementation

Windows clearly distinguishes between a **process** and a **thread**.

- A **process** is a container that provides:
 - Virtual address space
 - Handles to system resources
 - Security context
- A **thread** is the basic unit of execution within a process.

Windows uses a **one-to-one threading model**, where each user thread maps to one kernel thread.

Threads are created using the **Win32 API `CreateThread()`**, or indirectly through higher-level APIs like the Windows thread pool.

A Thread context includes all information needed to pause and resume a thread. It contains:

- **Thread ID**: A unique identifier for the thread.
- **CPU register set**: Values currently in CPU registers (e.g., program counter, stack pointer). This is the exact point where the thread is executing.
- **User-mode stack**: Used for function calls, local variables, and normal program execution.
- **Kernel-mode stack**: Used when the thread enters the OS kernel (e.g., system calls, interrupts).
- **Private data (libraries/DLLs)**: Thread-specific data used by runtime or dynamic libraries.

Windows maintains thread information in both user and kernel space using data-structures:

Structure	Location	Contents
ETHREAD (Executive Thread Block)	Kernel Space	Links thread to its process, points to KTHREAD.

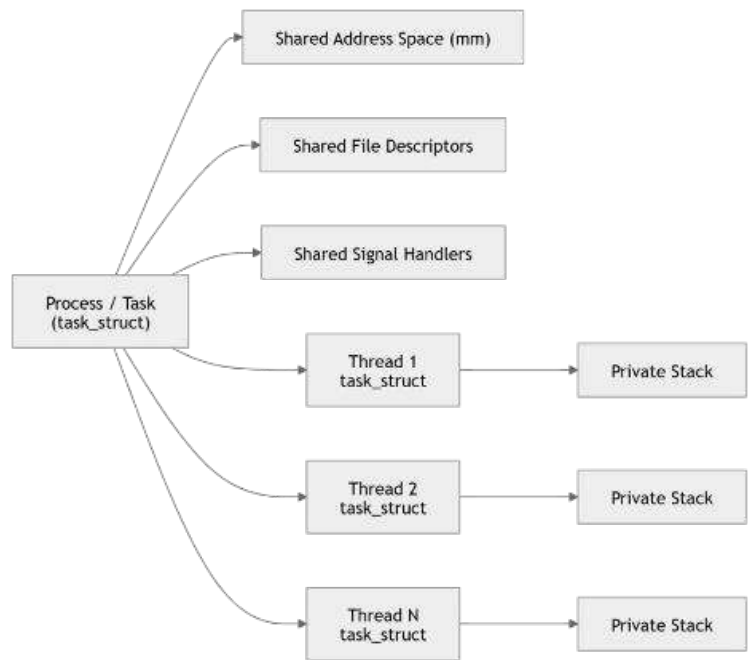
KTHREAD (Kernel Thread Block)	Kernel Space	Contains scheduling information, kernel stack, synchronization data
TEB (Thread Environment Block)	User Space	Thread ID, user stack, thread-local storage (TLS).

Comparison Table: Linux vs Windows Threads

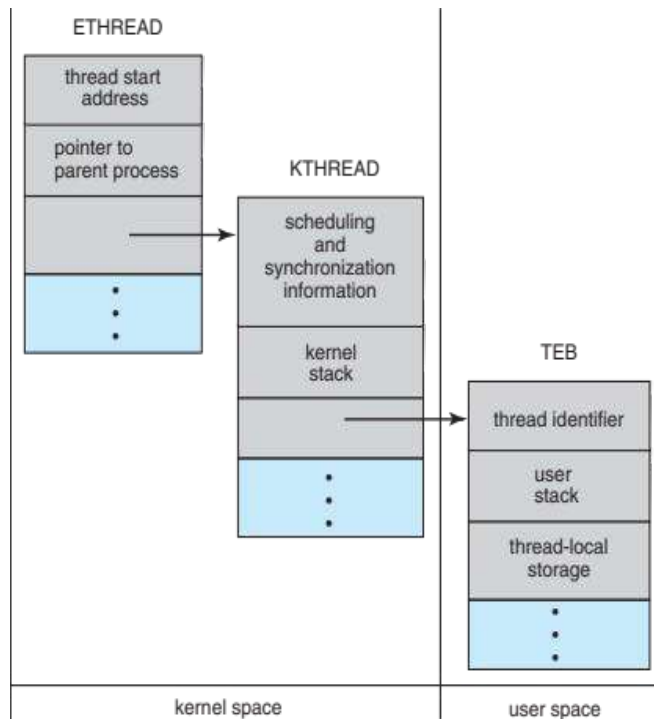
Feature	Linux	Windows
Thread Model	No strict separation between process and thread (both are tasks)	Clear distinction between process and thread
Internal Representation	task_struct represents both process and thread	Separate thread objects within process.
Creation System Call	clone() with share flags	CreateThread() (Win32 API)
User-Kernel Mapping	One-to-one (each task is a kernel thread)	One-to-one
Thread Library	Pthreads (POSIX)	Win32 threads API
Data Structures	task_struct has all thread info	ETHREAD (kernel), TEB (user), KTHREAD (kernel block)
Scheduling	Completely Fair Scheduler (CFS)	Priority-based preemptive scheduler
Fiber Support	Kernel threads + user threads via libraries	Supports fibers (user-level lightweight threads)

Block Diagrams

Linux Task Structure:



Windows Task Structure:



Part (b): POSIX Threads (Pthreads)

Pthreads (POSIX Threads) is a standardized C library defined by (IEEE 1003.1c) for creating and managing threads in UNIX-based systems such as Linux, macOS and others. It provides a low-level API for multithreading.

Key Pthread Functions

Function	Purpose
<code>pthread_create()</code>	Create a new thread and starts execution of a function. Arguments: thread ID, attributes, function pointer, argument.
<code>pthread_join()</code>	Wait for a thread to finish execution (like <code>wait()</code> for processes).
<code>pthread_exit()</code>	Terminate the calling thread.
<code>pthread_cancel()</code>	Request cancellation of another thread.
<code>pthread_mutex_lock()</code>	Acquires a mutex lock for critical section access.
<code>pthread_mutex_unlock()</code>	Release a mutex lock (exit critical section).
<code>pthread_attr_init()</code>	Initializes thread attributes (priority, stack size, etc.).
<code>pthread_detach()</code>	System clean up a thread automatically when it finishes.

Pthreads Example Flow

```

#include <pthread.h>
void *runner(void *param) {    // Thread work here    pthread_exit(0); }
int main() {
    pthread_t tid;
    pthread_attr_t attr;
    pthread_attr_init(&attr);          // init attributes
    pthread_create(&tid, &attr, runner, NULL); // create thread
    pthread_join(tid, NULL);            // wait for thread    return 0; }

```

Pthreads Execution Flow

- `pthread_create()` creates a new thread and starts execution of a specified function.
- Each thread runs independently but shares process memory with other threads.
- `pthread_join()` ensures the main program waits for thread completion.
- Synchronization is handled using mutexes and other thread synchronization tools.
- On Linux, Pthreads calls `clone()` internally which creates kernel-level threads.

Technique	API Methods
Extend Thread	<code>start()</code> , <code>run()</code> , <code>join()</code> , <code>sleep()</code>
Implement Runnable	<code>start()</code> , <code>run()</code>

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread using Thread class is running!");
    }
}

class MyTask implements Runnable {
    public void run() {
        System.out.println("Thread using Runnable is running!");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        Thread t2 = new Thread(new MyTask());

        t1.start();
        t2.start();

        System.out.println("Main method finished");
    }
}

```

Part (c): Implicit vs Explicit Threading

Explicit Threading

Explicit threading means the programmer manually creates, manages, and controls threads.

Features:

- Programmer creates threads using APIs such as `pthread_create()`, `CreateThread()`, or Java `Thread`.
- Programmer is responsible for synchronization, scheduling logic, and thread termination.
- Provides maximum control over thread behaviour.

Advantages:

- Full control over thread execution.
- Suitable for fine-grained performance tuning.

Disadvantages:

- Complex to manage.
- High risk of race conditions, deadlocks, and synchronization bugs.
- Difficult to scale for large applications.

Examples:

- Pthreads (C/C++)
- Java `Thread` class
- Win32 API threads

Implicit Threading

Implicit threading means the programmer only writes the task logic, while the system (compiler, runtime library, or framework) automatically creates, manages, and schedules threads. The programmer does not directly handle thread creation or management directly.

Features of Implicit Threading

- Threads are created and managed automatically by the compiler or runtime system.
- The programmer focuses only on defining the task, not thread management.
- The system decides the number of threads based on available hardware resources (e.g., CPU cores).
- It provides automatic load balancing and scheduling across threads.
- It is widely used in modern parallel frameworks such as OpenMP, thread pools, and GCD.

Advantages of Implicit Threading

- Reduces programming complexity and effort.
- Minimizes synchronization errors such as race conditions and deadlocks.
- Automatically scales with multicore processors.
- Improves productivity by hiding low-level thread management.

Disadvantages of Implicit Threading

- Less control over thread behavior and scheduling.
- Difficult to fine-tune performance for specific cases or workloads.
- Debugging internal thread behavior is harder because it is managed by runtime systems.

Three Key Implicit Threading Approaches

1. Thread Pools

A thread pool is a collection of pre-created threads that are reused to execute multiple tasks.

Features:

- Threads are created at startup and kept alive in a pool.
- When a task arrives, it is assigned to an idle thread from the pool.
- After execution, threads return to the pool instead of being destroyed.

Advantages:

- Reduces overhead of thread creation and destruction, since threads are reused.
- Improves performance by avoiding frequent OS-level thread management.
- Limits number of active threads, which helps prevent system overload and improves stability.

2. OpenMP

OpenMP is a compiler-based API for shared-memory parallel programming in C/C++/Fortran using directives.

- Programmer adds directives like `#pragma omp parallel`.
- The compiler/runtime automatically creates and manages threads.
- Best suited for **data parallelism**.

Example:

- `#pragma omp parallel for` distributes loop iterations across threads.

3. Grand Central Dispatch (GCD)

GCD is Apple's concurrency framework used in macOS and iOS. Programmer submits tasks (blocks) to queues and system manages threads automatically using a thread pool.

Two types of queues:

- **Serial Queue:** Executes tasks one at a time.
- **Concurrent Queue:** Executes multiple tasks in parallel.

GCD dynamically adjusts thread usage based on system load.

Which is Better for Programmer?

Feature	Explicit Threading	Implicit Threading
Thread Creation	Threads are created manually by the programmer.	Threads are created automatically by the compiler or runtime system.
Control	Full control over thread creation, execution, and termination.	Limited control; system decides thread management.
Programming Effort	High, because programmer manages all thread details.	Low, because programmer writes mostly sequential code.
Complexity	High complexity due to manual synchronization and management.	Low complexity since runtime handles threading.
Error Possibility	High risk of race conditions, deadlocks, and synchronization errors.	Lower risk because system manages synchronization and scheduling.

Performance Tuning	Fine-grained optimization is possible.	Limited tuning but generally optimized by runtime.
Scalability	Difficult to scale for large applications.	Automatically scales with number of CPU cores.
Examples	Pthreads, Java Threads, Win32 Threads.	OpenMP, Thread Pools, Grand Central Dispatch (GCD).
Best Use Case	When precise control over threads is required (e.g., real-time systems).	When parallelizing loops, data processing, and general applications.

Implicit threading is generally preferred for most programmers because it simplifies parallel programming and reduces programming errors.

- It reduces complexity, as the programmer focuses on the task logic rather than thread creation and management.
- It is less error-prone because thread creation, scheduling, and load balancing are handled by the runtime system.
- It improves scalability as the runtime can automatically utilize available CPU cores and adapt to different hardware configurations.
- Modern high-level frameworks such as Java ExecutorService, OpenMP, and Grand Central Dispatch (GCD) widely use implicit threading for better productivity and performance.